Design Pattern Leader and Followers, Heart Beat, Majority Quorum, Generation Clock

Leader and Followers

Intento e definizione

L'intento del pattern è avere un singolo server (il leader) che coordina la replicazione dei dati per tutto l'insieme di server (il cluster).

Il **problema** nasce dalla necessità di garantire la consistenza dei dati replicati su più server per ottenere fault tolerance. Il semplice utilizzo di un quorum di maggioranza non è sufficiente a garantire una consistenza forte in tutti gli scenari di fallimento.

→ *esempio di fallimento*: se uno scrittore aggiorna un dato ($x = 2$) solo sul Server1 (e fallisce sugli altri), e poi il Server1 "cade" dopo che un Client1 ha letto il nuovo dato, un successivo Client2 potrebbe leggere il vecchio dato ($x = 1$) dagli altri server. Questo crea un'inconsistenza in cui una lettura successiva vede uno stato precedente a una lettura passata.

La **soluzione** prevede di assegnare ruoli specifici ai server del cluster:

1. **Leader**: viene eletto un unico server responsabile di prendere tutte le decisioni per l'intero cluster e di propagarle agli altri.
2. **Followers**: gli altri server che seguono le decisioni del Leader.

Gestione delle richieste

Tutte le richieste dei client devono essere gestite dal Leader. Se un client invia una richiesta a un server che è un Follower, questo deve inoltrarla al Leader.

Elezione del Leader

Un sistema deve essere in grado di scegliere un Leader in due momenti: all'avvio del cluster o quando il Leader corrente fallisce.

- **Stati del server**: un server può trovarsi in uno dei seguenti stati → Leader, Follower, o Candidato (cioè cerca Leader).
- **Procedura**: all'avvio, ogni server cerca un Leader. Se non lo trova, avvia un'elezione del Leader. Durante l'elezione, il cluster non accetta richieste.
- **Criteri di scelta**: solitamente vince il server con i dati più aggiornati (basandosi sul Generation Clock o sull'indice di log). A parità di merito (tutti i server sono equamente aggiornati), si usano criteri come l'ID più alto o chi ha richiesto il voto per primo (come nell'algoritmo RAFT, *Reliable Replicated Redundant and Fault-Tolerant*)

Meccanismi di supporto

Il funzionamento corretto del pattern si basa su due concetti tecnici fondamentali:

1. **Heart Beat** (battito cardiaco): il Leader invia periodicamente un messaggio agli altri server per indicare che è attivo (garantire liveness). Se i Follower non ricevono il "*battito cardiaco*" del Leader entro un certo intervallo, deducono che il Leader è fallito e avviano una nuova elezione.

2. **Generation Clock** (orologio di generazione): è un numero che cresce in modo monotono e serve ad identificare l'epoca del Leader corrente.
    - viene incrementato ad ogni nuova elezione.
    - serve a risolvere il problema dei "vecchi leader" (o server zombie): se un vecchio Leader disconnesso torna online, il cluster può rifiutare le sue richieste notando che il suo generation clock è inferiore a quello attuale.

Heart Beat

Intento e problema

L'intento del meccanismo di Heart Beat è mostrare che un server è disponibile (liveness) inviando periodicamente un messaggio a tutti gli altri server del cluster.

Il problema nasce dalla necessità di accorgersi prontamente del fallimento di un nodo (ad esempio, il Leader) per poter avviare azioni correttive, come l'elezione di un nuovo leader o la ridistribuzione delle responsabilità sui dati.

Soluzione e funzionamento

- **Invio periodico**: il server invia una richiesta periodica agli altri nodi per segnalare la sua presenza.
- **Intervalli di tempo**: bisogna configurare attentamente le tempistiche:
    - l'**intervallo** di invio dell'**heart beat** deve essere maggiore del tempo di **round-trip** (andata e ritorno) della rete.
    - il **timeout** (tempo dopo il quale un server è considerato fallito se non risponde) deve essere un multiplo dell'intervallo di heart beat.
    - *regola*: timeout > intervallo heart beat > intervallo round-trip.
    - *esempio*: round-trip=20 ms, heart beat = 100 ms, timeout = 1 s.
- Compromesso: un intervallo di heart beat molto breve permette di rilevare velocemente i guasti, ma aumenta la probabilità di falsi positivi (con il rischio di rilevare un fallimento che in realtà è solo un ritardo di rete).

Quorum di maggioranza (majority quorum)

Intento e problema

L'intento è evitare che, in caso di partizione della rete, due gruppi diversi di server possano prendere decisioni indipendenti e conflittuali (problema noto come *split-brain*). Si richiede che ogni decisione sia approvata dalla maggioranza dei nodi.

Il problema di fondo è bilanciare la proprietà di Safety (il sistema è sempre in uno stato corretto) e la proprietà di Liveness (il sistema progredisce sempre) quando si replicano i dati.

Calcolo e tolleranza ai guasti

- **Regola della maggioranza**: un aggiornamento è considerato valido solo se confermato da $\frac{n}{2} + 1$ nodi, dove n è la dimensione del cluster.
- **Dimensionamento**:

- per tollerare f fallimenti, è necessario un cluster di dimensione $2f + 1$.
- *esempio*: con 3 server, il quorum è 2 e si tollera 1 fallimento;
  con 5 server, il quorum è 3 e si tollerano 2 fallimenti.

- Impatto sulle prestazioni: ogni server aggiunto al quorum rallenta le operazioni di scrittura, riducendo il *throughput* (quantità di richieste totali gestite nel tempo).

Limiti e flessibilità

- **Non sufficienza**: il majority quorum da solo non è sufficiente a garantire una consistenza forte in tutti gli scenari di fallimento dei server (motivo per cui si introduce il pattern *Leader and Followers*), dove quindi i client vedrebbero dei dati inconsistenti.
- **Quorum flessibili**: è possibile definire quorum di dimensioni diverse per operazioni diverse (es. lettura e scrittura), a condizione che ci sia un'intersezione non vuota tra i gruppi di server coinvolti (per garantire la consistenza, così che almeno un nodo letto contenga l'ultima scrittura).
  Un'operazione frequente (tipicamente la lettura) potrebbe avere un quorum di dimensione più piccolo per renderla più veloce.
  - *esempio*: in un cluster di 5 nodi, il quorum di lettura potrebbe essere 2 mentre il quorum di scrittura 4. L'intersezione è quindi non vuota.

Generation Clock

Intento e definizione

Il Generation Clock è un numero che cresce in modo monotono (aumenta sempre) e serve ad indicare la "generazione" o l'epoca attuale di un server. Il suo scopo principale è permettere al sistema di distinguere tra il Leader attuale e un vecchio Leader che potrebbe essere stato disconnesso temporaneamente.

Il problema è che in un sistema Leader and Followers, può accadere che il Leader corrente perda la connessione con i Follower (ad esempio per problemi di rete o una pausa del processo), pur continuando a funzionare localmente.

1. Durante la sua assenza, il cluster elegge un **nuovo Leader**.
2. Quando la rete torna disponibile, il **vecchio Leader** (che non sa di essere stato sostituito) invia richieste di replicazione ai Follower.
3. Il sistema deve essere in grado di rilevare che queste richieste provengono da un componente obsoleto e quindi ignorarle.

Soluzione

Il Generation Clock risolve il problema attraverso il seguente meccanismo:

1. **Incremento**: ogni volta che avviene una nuova elezione del Leader, il numero della generazione viene incrementato.
2. **Persistenza**: il valore della generazione deve essere salvato in modo che sia disponibile anche al riavvio del server (letto dal log).
3. **Comunicazione**:
   - i server si scambiano la generazione durante le richieste di voto.

    - il Leader include il generation clock corrente in ogni richiesta inviata ai Follower e nei messaggi di heart beat.
4. **Verifica**: se un server riceve una richiesta contenente un generation clock inferiore al proprio (più vecchio), capisce che proviene da un leader decaduto e la rifiuta.

Esempio pratico

- esiste un **Leader1** con generazione 1.
- Leader1 si ferma per 5 secondi (pausa/disconnessione).
- nel frattempo, il cluster elegge un nuovo Leader e la generazione sale a 2.
- Leader1 torna attivo e invia richieste con generazione 1.
- i Follower e il nuovo Leader respingono le richieste perché la generazione 1 è obsoleta (più vecchia) rispetto alla 2.
- Leader1 riceve la risposta di fallimento, capisce di non essere più il capo, aggiorna la sua generazione a 2 e diventa un **Follower**.